PROBLEM SPECIFICATION

# Carbon Receipts

Traceable carbon figures for companies and the funds that hold them.

Every carbon figure comes with a receipt: the report, page and quoted text it came from.

**Leo McGinn** · Version 1.1 · 10 October 2026

CONTENTS


| | |
|---|---|
| 1 | Project definitions and users |
| 2 | Contextual analysis and project rationale |
| 3 | Technical strategy and requirements |
| 4 | High-level work plan |
| Appendix | Initial product backlog |

# 1 Project definitions and users

## 1.1 Project title

> Carbon Receipts: traceable carbon figures for companies and the funds that hold them.

## 1.2 Problem statement

UK listed companies have to publish their greenhouse gas emissions every year. Since 2019, quoted companies have reported their Scope 1 emissions (from fuel and processes they control) and Scope 2 emissions (from the electricity and heat they buy) in their annual report, alongside the previous year's figures, under the Streamlined Energy and Carbon Reporting rules. Funds then roll these figures up into carbon metrics for their whole portfolio. EU fund managers report a carbon footprint and the greenhouse gas intensity of the companies they hold under the Sustainable Finance Disclosure Regulation, and large UK pension schemes publish climate reports with similar figures.

Checking any of those numbers is hard. The emissions sit somewhere in a 200-page PDF, in a table, a chart or a paragraph, and the place changes from company to company and year to year. The figures aren't directly comparable either: Scope 2 can be measured two ways (location-based or market-based), financial years don't line up, and companies restate earlier figures. Researchers who collect them describe the work as a tedious manual process.

Fund-level figures usually come from commercial data providers. Even council pension funds buy them in. Where a company hasn't reported, providers fill the gap with modelled estimates, and nothing links a figure back to the page it came from. A pension saver, journalist or researcher has no practical way to see what's behind a company's or a fund's carbon figure, or how much of it was reported rather than estimated.

## 1.3 System overview and user profile

My proposed solution is a data pipeline that produces carbon figures for UK listed companies, and for the funds that hold them, where every number comes with a receipt: the report, page and quoted text it came from. The user chooses the companies, either by entering names or tickers or by loading a fund's published list of holdings. For each company, the pipeline finds its latest annual report in the FCA's National Storage Mechanism, the official free store of UK listed company filings. It picks out the pages most likely to hold emissions and revenue, and an AI model extracts the figures. Rules flag anything that looks wrong, and a person reviews each figure beside an image of its source page before it can be used. Every review decision is logged.

Approved figures go into a data warehouse, which works out each company's carbon intensity (emissions per million of revenue). When the companies come from a fund, it also works out the fund's carbon intensity, weighted by holding size, its share of each company's emissions where enough data exists, and its coverage: how much of the fund, by value, has reported figures behind it. A Power BI report shows each company's figures with

links to their source pages, and lets a fund's user go from the fund's figure to a company and on to the page its number came from. I'll also measure how often the AI gets figures right against a set I've checked by hand, and publish the result.

The users are people who want to check carbon claims for themselves. Investors want to look up the companies they care about, or load the fund in their pension or ISA, and see clear figures with their sources, without technical knowledge. Researchers, journalists and campaigners want company figures they can cite and an honest measure of accuracy. The reviewer, which in this project is me, needs to check figures quickly with the evidence on screen. The first version covers any UK listed company, tested on the companies in a FTSE 100 tracker and one more UK equity fund. It won't estimate emissions for companies that don't report them, break Scope 3 down by category, cover reports from outside the UK, or give investment advice.

# 2 Contextual analysis and project rationale

## 2.1 Current context and existing solutions

Today most people either copy figures out of reports by hand or rely on someone who has paid for them. Manual collection is slow and hard to audit, because the source of each number ends up in a spreadsheet comment or someone's memory. The commercial alternative is data from ESG providers such as MSCI, Sustainalytics, LSEG and Bloomberg, which is expensive, mixes reported figures with estimates, and doesn't show users the page behind a figure. CDP collects climate data directly from companies through a questionnaire, but its detailed data is for signatories or paid licences, and the answers are self-reported rather than taken from the annual report.

Some free tools exist. MSCI's public fund search shows a fund's weighted average carbon intensity and how much of it has data, but it's a headline number built partly on MSCI's own estimates, and nobody outside can check the company figures underneath. The LSE's Transition Pathway Initiative publishes free, source-based assessments, but only for selected high-emitting sectors and with a focus on future pathways rather than a fund's current footprint. Climate TRACE offers free independent estimates from satellites and models, which are useful for comparison but aren't what companies report.

AI is already being used to read the reports. Commercial products such as Briink and Tracenable extract ESG data from documents with citations and human review, but they're closed, sold to companies and investors, and the only accuracy figures I could find come from their own internal tests. In open source, climatextract, from LMU Munich and Deutsche Bundesbank, uses an LLM to pull Scope 1 to 3 figures from sustainability reports into CSV files with page numbers. It stops at extraction, with no review step, checks, warehouse, fund metrics or published accuracy. OS-Climate's extraction tools are archived. None of these takes a chosen list of companies, or a fund's holdings, through to reviewed figures that can each be traced to a page, which is the gap Carbon Receipts fills.

## 2.2 Benefits of the proposed system

Anyone can look up a company and see its emissions, revenue and carbon intensity, each with the report and page it came from. Investors who load their fund also see the fund's carbon intensity, how much of the fund that covers, and the receipt behind every company in it. Gaps are shown as gaps, so a fund can't look cleaner just because its biggest emitters didn't report.

Researchers and journalists get company figures with sources they can cite, exportable as CSV, along with a published accuracy rate, so they know how far to trust the data before they use it. The review app means a person signs off every figure, and the audit log shows who changed what and why.

Because it runs on free tiers and an ordinary laptop, anyone can rerun it for another fund or a new year of reports, and the code and accuracy results are public.

### 2.3 Initial risk identification

Firstly, the AI model's free tier has low daily limits that change without notice. A full run means hundreds of requests, so hitting a limit mid-run is likely. To mitigate this, I will process reports through a job queue that retries with back-off, caches every result so nothing is extracted twice, and can stop and resume across days. The model sits behind a small interface, so I can switch provider if the free tier disappears.

Secondly, extraction accuracy may fall short, especially where a report shows its emissions table as an image. To mitigate this, I will send page images to the model for those pages, run rule checks before review, have a person approve every figure, and publish the accuracy I actually measure, whatever it is.

Moreover, reports vary in units, Scope 2 method, reporting period and restated figures, which could make comparisons misleading. I will normalise units to tonnes of CO2 equivalent, store the method and period with each figure, and flag year-on-year jumps for review. A related risk is that enterprise value (a company's market value plus its debt), which the fund's share of emissions depends on, is harder to get for free than revenue. Carbon intensity, which only needs revenue, is therefore the core metric, and the share-of-emissions figures are a stretch goal with their own coverage figure.

Finally, the timeline is tight: seven weekly sprints to a working version by the end of November. I will build the must-have stories first, drop should-haves before must-haves if time runs short, and leave the extras until after the main version is public.

## 3 Technical strategy and requirements

### 3.1 High-level requirements

| Category | Requirement statement (the system must…) | Justification / scope definition |
|---|---|---|
| FR1 | The system must accept the companies to check, either entered by name, ticker or identifier, or loaded from a fund's published holdings file with each holding's weight, market value and as-of date. | Defines which companies are checked. A fund's holdings also supply the weights for the fund metrics. |
| FR2 | The system must find each company's latest annual report in the FCA's National Storage Mechanism and store it with its company, period, source URL and checksum. | Defines the evidence. The checksum stops reports being downloaded twice. |
| FR3 | The system must process reports through a job queue with retries, back-off and handling of daily rate limits. | Runs must survive free-tier limits and crashes. |

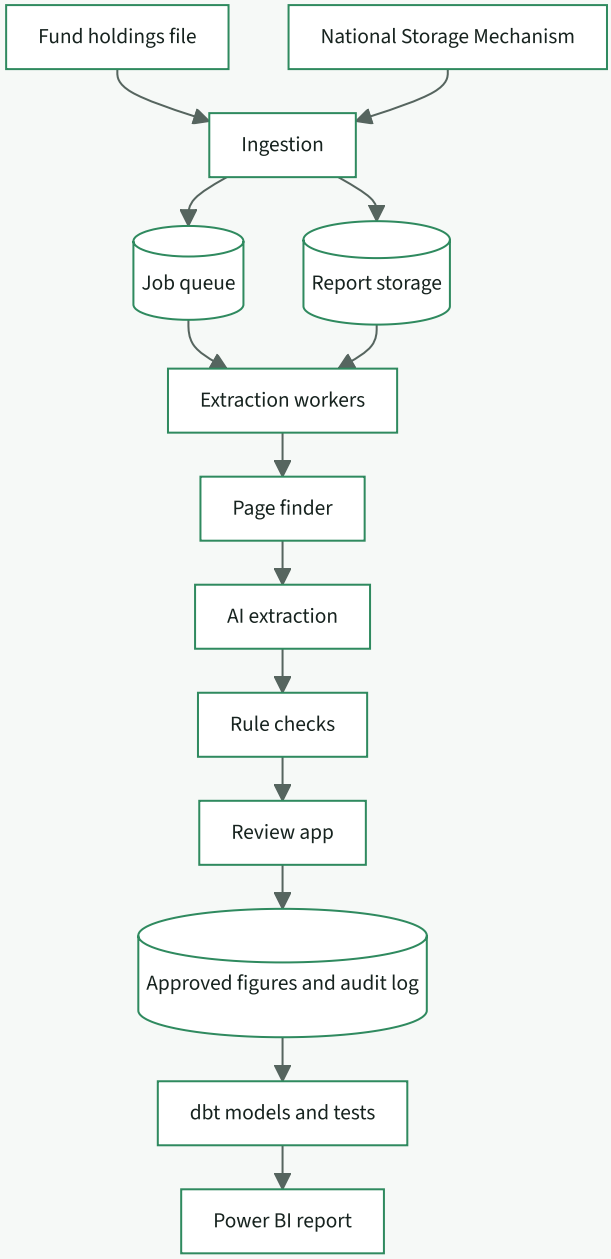
| Category | Requirement statement (the system must…) | Justification / scope definition |
|---|---|---|
| FR4 | The system must extract every page's text, render page images on demand, and rank the pages most likely to hold emissions and revenue. | Only a few pages of each report go to the AI model, which saves requests and improves accuracy. |
| FR5 | The system must extract Scope 1, Scope 2 (each method reported), the Scope 3 total, revenue, energy use, units and reporting period, for the latest and prior year, as structured output. | Defines the figures in scope. Scope 3 by category is out of scope. |
| FR6 | The system must store a receipt with every figure: document, page, quoted text, model, prompt version and time. | The core promise of the project. Anyone can check any figure. |
| FR7 | The system must check every figure with rules (units, plausible ranges, scope totals, reporting period, year-on-year change), and should compare revenue with the report's tagged (iXBRL) value. | Catches errors before review. The revenue cross-check is a should-have. |
| FR8 | The system must provide a review app showing each figure beside its source page, with approve, correct and reject actions. | A person signs off every figure before it's used. |
| FR9 | The system must log every review decision (reviewer, time, old value, new value, reason) in a log that can't be edited. | Makes each figure's history auditable. |
| FR10 | The system must model companies, holdings and approved figures as a star schema, calculate each company's carbon intensity, and calculate each fund's carbon intensity and coverage. It should also calculate a fund's financed emissions and carbon footprint where enterprise value is available. | The company-level and fund-level outputs. Footprint is a should-have, because enterprise value data may be incomplete. |
| FR11 | The system must provide a Power BI report with a company view that links to receipts, a fund overview that drills down to companies, and a data quality page. | How investors and researchers use the results. |
| FR12 | The system must measure extraction accuracy against a hand-checked evaluation set and publish the result. | Users can judge how far to trust the figures. |
| FR13 | The system must run on Azure, with reports in Blob Storage and the warehouse in Azure SQL Database. | A cloud deployment that stays free under Azure SQL's free offer. |
| FR14 | The system should support more than one fund through configuration, and export approved figures with their receipts as CSV. | Lets users check their own fund and reuse the data. Should-haves. |
| FR15 | The system could answer plain-English questions about companies and funds using only approved figures, citing their receipts. | Stretch goal after the main version. |
| FR16 | The system could compare reported emissions with Climate TRACE estimates and flag large gaps, and deploy its models to a Microsoft Fabric warehouse. | Stretch goals, to be considered at the end. |

| Category | Requirement statement (the system must…) | Justification / scope definition |
|---|---|---|
| NFR1 Accuracy | At least 85% of Scope 1, Scope 2 and revenue values must be correct before human review, measured on a hand-checked set of at least 30 reports. | A measurable target for the AI step, published with the project. |
| NFR2 Coverage and transparency | Approved, receipted Scope 1 and 2 figures must cover at least 90% of the FTSE 100 fund by value. Missing figures must be shown as missing, never estimated. | The fund figure has to rest on reported data, and the gaps have to be visible. |
| NFR3 Cost and hardware | The system must run on a Windows laptop with 8 GB of RAM and no GPU, using free tiers only. | Anyone can run it, at no cost. |
| NFR4 Reliability | Re-running must be safe: unchanged reports aren't downloaded or extracted again, and a stopped run resumes where it left off. | Needed to work within daily rate limits. |
| NFR5 Reproducibility | Every figure must record the model, prompt version and configuration that produced it. | Results can be explained and repeated. |
| NFR6 Security | No secrets in the repository. Report PDFs aren't committed, and public outputs hold links, page numbers and short quotes only. | Protects API keys and respects the reports' copyright. |
| NFR7 Maintainability | Unit tests and dbt tests must run in CI on every push, and the README must cover setup, architecture, metric definitions, accuracy and limitations. | Keeps the pipeline correct as it changes, and lets others run it. |
| NFR8 Performance | A 300-page report should be processed in under 2 minutes on the laptop, not counting time waiting for the AI model. | Keeps a full run practical. |

## 3.2 Proposed technology stack

| Technology / tool | Category (e.g. backend, database) | Technical rationale (links to specific requirements) |
|---|---|---|
| **Python 3.12, managed with uv** | Language and pipeline | Good libraries for HTTP, PDFs and data, and uv gives reproducible environments. Supports FR1 to FR7 and NFR5. |
| **Fund holdings file, FCA National Storage Mechanism, tagged (iXBRL) filing data** | Data sources | Free, public and official. Supports FR1, FR2 and FR7. |
| **httpx and fsspec** | Ingestion and file storage | Downloads files and APIs, and lets the same code write to a local folder or Azure Blob Storage. Supports FR2 and FR13. |
| **pypdfium2 and pdfplumber** | PDF processing | Fast and CPU-only, for page text, page images and tables, with licences suitable for a public repo. Supports FR4 and NFR8. |

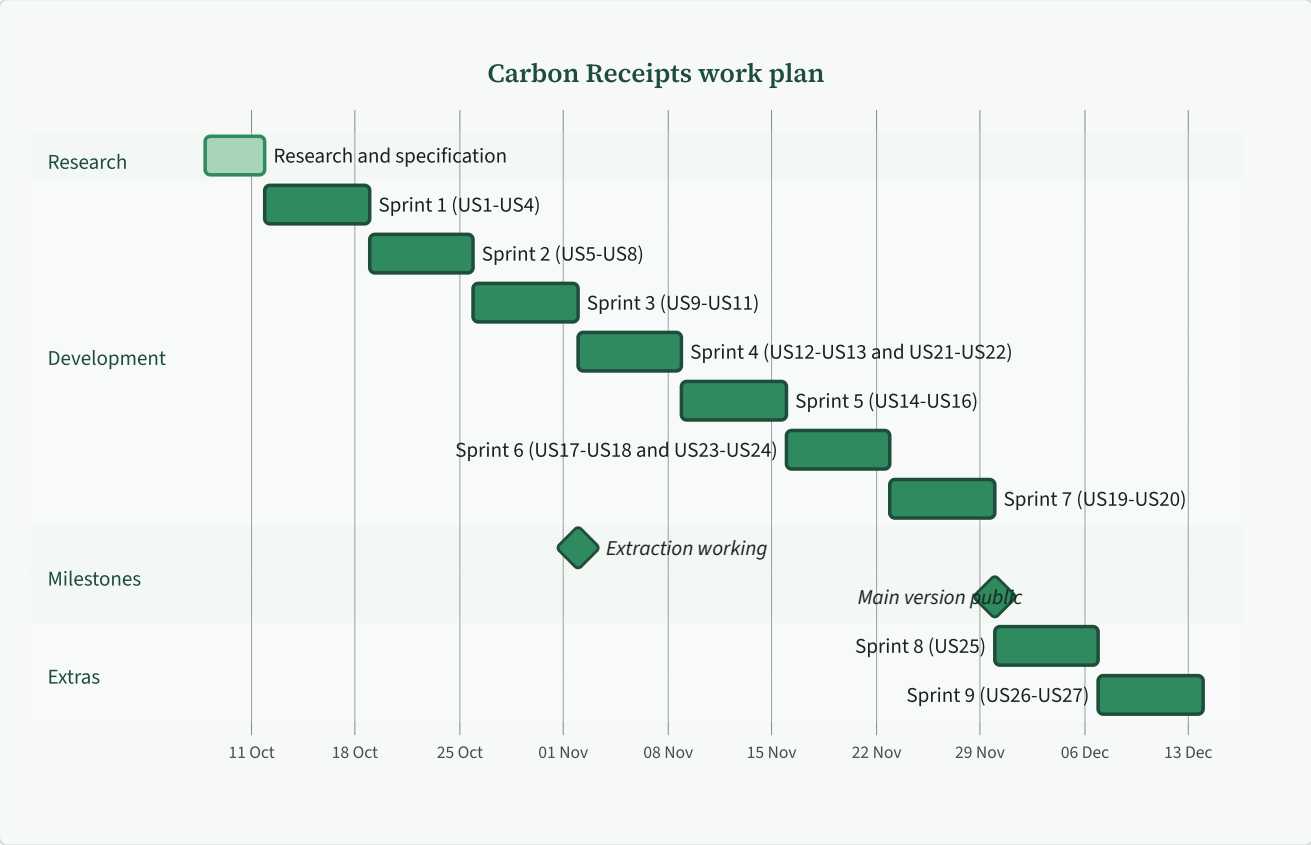
| Technology / tool | Category (e.g. backend, database) | Technical rationale (links to specific requirements) |
|---|---|---|
| **BM25 keyword search and fastembed embeddings** | Page finding | Emissions tables use predictable terms, and small local embeddings catch unusual wording, without a vector database or GPU. Supports `FR4` and `NFR3`. |
| **Gemini API (free tier) with Pydantic** | AI extraction and validation | Reads page text and images and returns JSON in a fixed schema, which Pydantic validates. Free. Supports `FR5`, `FR7`, `NFR1` and `NFR3`. |
| **SQL Server 2025 Express, then Azure SQL Database** | Database and warehouse | Free in both places, T-SQL in both, and a native Power BI connection. Express caps its own memory, which suits the laptop. Holds the job queue too. Supports `FR3`, `FR10`, `FR13` and `NFR3`. |
| **dbt Core (dbt-sqlserver)** | Data transformation | Version-controlled, tested SQL for the star schema and fund metrics, with generated documentation. Supports `FR10` and `NFR7`. |
| **Streamlit** | Review app | A quick way to build a Python web app that shows a page image beside each figure. Supports `FR8` and `FR9`. |
| **Power BI Desktop** | Reporting | Interactive drill-down from fund to company to receipt. Supports `FR11`. |
| **pytest, ruff and GitHub Actions (with SQL Server in a container)** | Testing and CI | Lint, unit tests and `dbt build` on every push, with no Docker needed on the laptop. Supports `NFR7`. |
| **Azure Blob Storage and Azure SQL Database (free offer)** | Cloud deployment | A deployment that stays within free limits, with auto-pause when they run out. Supports `FR13` and `NFR3`. |




*Planned pipeline*

# 4 High-level work plan

I'll use an agile approach in weekly sprints, starting on 12 October 2026. Each sprint covers the design, build and testing of its user stories. I'll track progress through sprint reviews, a short weekly reflection and the project's GitHub Projects board, and every commit, issue and pull request names its story ID. The aim is a working, public version by the end of November, with the must-have and should-have stories done. The extras follow in early December.



APPENDIX

# Initial product backlog

| Priority | User story | Description / acceptance criteria |
|---|---|---|
| US1 MUST | As a maintainer, I want a repo with uv, ruff, pytest and a CI workflow so that every change is checked. | Lint and tests run on every push, and the result shows on the commit. |
| US2 MUST | As a maintainer, I want SQL Server Express and a dbt project connected, with CI running dbt against a SQL Server container, so that models are tested on every push. | `dbt build` succeeds locally and in CI from a fresh clone. |
| US3 MUST | As an investor, I want to enter the companies I care about by name or ticker, or load my fund's holdings, so that I can check exactly the companies I'm interested in. | Entered names and tickers resolve to companies, and any that can't be matched are listed. A fund's holdings load with identifier, name, weight, market value and as-of date, and a changed column layout stops the load with a clear error. |
| US4 MUST | As a maintainer, I want each company matched to its latest annual report so that every company has a source document. | Each company has a report link, or appears in an unmatched list with a reason. |
| US5 MUST | As a maintainer, I want reports stored with metadata and a checksum so that nothing is downloaded twice. | A second run doesn't download reports already stored with the same checksum. |
| US6 MUST | As a maintainer, I want reports processed through a job queue with retries and back-off so that runs survive rate limits and crashes. | A rate-limited job is retried later, and after a restart the run continues where it stopped. |
| US7 MUST | As a maintainer, I want each report's page text and images extracted so that later steps can search and show pages. | Text for every page is stored, and any page can be shown as an image. A 300-page report takes under 2 minutes. |
| US8 MUST | As a researcher, I want a hand-checked set of at least 30 reports so that accuracy can be measured honestly. | True figures and page numbers for 30 reports are stored as a versioned file in the repo. |
| US9 MUST | As a maintainer, I want the pages most likely to hold emissions and revenue ranked so that only a few pages go to the AI model. | The page with the Scope 1 and 2 figures is in the top 5 for at least 90% of the hand-checked reports. |
| US10 MUST | As a maintainer, I want the AI model to return emissions, revenue, energy use, units and reporting period as structured output so that figures load straight into tables. | Output passes schema validation, or the job fails with the reason recorded. |
| US11 MUST | As an investor, I want every figure to carry a receipt so that I can check it myself. | Document, page, quote, model and prompt version are stored with each figure, and the quote is found in that page's text. |
| US12 MUST | As a reviewer, I want rule checks run on every figure so that likely errors reach me already flagged. | Figures with the wrong unit, implausible values, scope totals out by more than 1% or big year-on-year jumps are flagged with the rule that failed. |
| US13 MUST | As a researcher, I want every review decision logged so that each figure's history can be audited. | Old value, new value, reviewer, time and reason are stored for each decision, and the log can't be edited. |

| Priority | User story | Description / acceptance criteria |
|---|---|---|
| US14 MUST | As a reviewer, I want each figure shown beside its source page with the quote highlighted, so that I can approve, correct or reject it quickly. | Approving or correcting a figure updates its status, logs the decision and loads the next figure. |
| US15 MUST | As a maintainer, I want holdings and approved figures modelled as a star schema with tests so that metrics are consistent and checked. | Facts and dimensions build with `dbt build`, and not-null, unique, relationship and accepted-value tests pass. Every figure in a fund metric has a receipt. |
| US16 MUST | As an investor, I want each company's carbon intensity, and my fund's weighted carbon intensity and coverage, calculated so that I know how carbon-heavy the companies are and how much of the fund that covers. | Company intensity, fund intensity and coverage match a hand calculation for 5 holdings. |
| US17 MUST | As an investor, I want a report with a company view, a fund overview and a data quality page so that I can go from any figure to its sources. | From a company's figure, its source page link is one click away. From a fund's figure, it's three. |
| US18 MUST | As a researcher, I want extraction accuracy measured and published so that I know how far to trust the figures. | Accuracy per field before review is reported in the README. |
| US19 MUST | As a maintainer, I want a README covering setup, architecture, metric definitions, accuracy and limitations, plus dbt docs, so that anyone can run and judge the project. | A new user can run the pipeline end to end by following the README. |
| US20 MUST | As a maintainer, I want the pipeline running on Azure so that it works in the cloud at no cost. | Reports land in Blob Storage, dbt builds in Azure SQL Database, and the database pauses at the free limit. |
| US21 SHOULD | As a reviewer, I want revenue checked against the report's tagged value so that revenue errors are caught automatically. | Differences over 1% from the tagged (iXBRL) value are flagged. |
| US22 SHOULD | As an investor, I want to add another fund by configuration so that I can check the fund I actually hold. | A second fund in the config appears in the warehouse and report alongside the first. |
| US23 SHOULD | As an investor, I want financed emissions and carbon footprint where enterprise value is available so that I can see my fund's share of emissions. | Footprint is calculated with its own coverage figure, and companies without enterprise value are counted as gaps. |
| US24 SHOULD | As a researcher, I want approved figures with their receipts exported as CSV so that I can use and cite them. | The CSV has values, units, periods, document links, pages and quotes. |
| US25 COULD | As an investor, I want to ask questions about companies or my fund in plain English so that I can explore them without the report. | Every figure in an answer is an approved figure with its receipt linked, and questions the data can't answer are refused. |
| US26 COULD | As a researcher, I want reported emissions compared with independent estimates so that large gaps stand out. | Gaps over a set threshold against Climate TRACE are flagged in the report. |
| US27 COULD | As a maintainer, I want the dbt models deployed to a Microsoft Fabric warehouse so that the project also runs on Fabric. | Models build and tests pass against a Fabric trial warehouse. |